

Parallel and Distributed Computing

Parsim MPI Report

João Gouveia - 102611, Rodrigo Arede - 102606, Gonçalo Rua - 102604

April 4, 2025

1 Introduction

This stage of the project focuses on parallelizing the serial version of a particles simulation, using OpenMP. This simulation models the gravitational interaction between particles in a 2D space, and the goal is to improve the computational efficiency through parallelization.

2 Approach

The 2D space is divided into a grid, where each cell's center of mass is used to approximate the gravitational interactions. We took the approach of parallelizing the cells of the grid, with the objective of each cell being processed concurrently with other cells. We also parallelized the generation of the initial grid which required calculations to attribute each particle to the corresponding cell where it's initially located.

3 Synchronization Concerns

We had some challenges with the synchronization of the different threads, specifically when we encountered problems with data dependencies between them. One clear example was the appending of newly updated particles to the cells, while other particles from different cells were being computed. To solve this problem we had to separate this stage in two, which led us to temporarily save all the new particles in a 'vector<vector<Particles>>', and only after all the particles in all the cells are done computing we proceed to add them to the correct cells.

4 Load Balancing

With OpenMP we used dynamic scheduling to evenly distribute computation across threads. This technique distributes workload among threads at runtime rather than assigning it stati-

cally before execution. Comparing the static vs dynamic approach, the latter gives us the best performance of the two.

5 Performance Results

5.1 Methodology

For analysing the performance of our implementation we used the following inputs:

Test Number	Input
1	12672 0.05 3 10 10
2	5893 0.05 3 10 10
3	8555 0.05 3 10 10
4	12 100 5 10000 10000
5	-11 3500 20 500000 10
6	1 5000 100 1000000 4
7	1 5000 100 1000000 100
8	1 5000 20 1000000 10
9	1 1000 3 10000 10000
10	3 5000 50 1000000 300
11	3 5000 50 1000000 500
12	-1 1000 30 100000 1000

5.2 Results

The table below shows the execution times (in seconds) for the serial and OpenMP versions of the program:

Test Number	1 thread (serial)	2 threads	4 threads	8 threads				
1	0.0s	0.0s	0.0s	0.0s	2	0.0s	0.0s	0.0s
2	0.0s	0.0s	0.0s	0.0s	3	0.0s	0.0s	0.0s
3	0.0s	0.0s	0.0s	0.0s	4	53.9s	31.4s	21.2s
4	53.9s	31.4s	21.2s	16.7s	5	49.7s	25.5s	13.6s
5	49.7s	25.5s	13.6s	7.5s	6	1.6s	1.0s	0.8s
6	1.6s	1.0s	0.8s	0.8s	7	34.9s	23.0s	18.1s
7	34.9s	23.0s	18.1s	15.7s	8	57.1s	29.7s	15.6s
8	57.1s	29.7s	15.6s	8.7s	9	245.9s	141.4s	89.8s
9	245.9s	141.4s	89.8s	68.0s	10	289.4s	160.5s	100.1s
10	289.4s	160.5s	100.1s	74.1s	11	481.9s	263.0s	171.8s
11	481.9s	263.0s	171.8s	128.5s	12	81.8s	45.7s	30.4s
12	81.8s	45.7s	30.4s	24.0s				

6 Conclusion

The project successfully implemented a parallel particles simulation, demonstrating significant speedups using OpenMP. The next step is to use MPI to parallelize the workload across different machines, and exploring an hybrid MPI+OpenMP approach for even better performance and scalability.